Python for HPC

SDSC Summer Institute 2026
Andrea Zonca
Friday, August 7 | 8:30 AM to 11:20 AM

SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego
HALICIOĞLU SCHOOL OF DATA SCIENCE AND COMPUTING

What you will be able to do

- Speed up a numerical loop with Numba and time it fairly.
- Use Numba threads to exploit the CPU cores on one node.
- Scale a Dask computation across multiple nodes.

How to follow the session

- Follow the action shown on the slide.
- Work in the notebook while the slide stays visible.
- Return here for the debrief and next step.
- Blue = stuck. Yellow = ready.

Today's path

- Setup and environment staging.
- Threads, the GIL, and Numba on one node.
- Dask task graphs and multi-node scaling.
- Recap and reusable workflows.

1 of 7 | Setup

- Time check: 8:30 AM.
- Goal: start environment staging and establish the parallelism model.
- We move on once the cached environment is ready.

SETUP | Update the lesson repository

- Open the repository: `cd ~/sdsc-summer-institute-2026`.
- Update to the latest lesson: `git pull`
- If pull is blocked by local changes: `git stash` → `git pull` → `git stash apply`

SETUP | Stage the Conda environment

- From the repository, `cd 6.1_python_for_HPC`.
- Run `sbatch 0_python_condaenv_scratch/python_expanse.slurm`
- Check with `squeue --me`. Continue while it runs.

SETUP | Why not Conda on shared storage?

- A Conda environment contains thousands of small files.
- Creating and importing them causes heavy metadata I/O.
- Parallel filesystems handle large I/O well; many tiny file operations are expensive.
- Node-local SSD avoids that metadata bottleneck during creation and imports.

SETUP | Why pack the environment?

- Solve and build the environment once — not once per job or node.
- `conda-pack` turns the full environment into one relocatable `.tar.gz` archive.
- Each job extracts that archive onto fast node-local scratch.
- `conda-unpack` fixes relocated paths after extraction.

SETUP | What the staging job does

- Builds `pythonhpc` from `environment.yaml` on node-local SSD.
- Uses the allocated CPU cores during solve, build, and packing.
- Caches one reusable archive in `~/.galyleo/pythonhpc/`.
- Galyleo and Dask stage `pythonhpc` from the cache — no rebuild.

PARALLELISM | Threads and the GIL

Threads
share memory
lightweight

Python code
runs in threads
inside one
interpreter

GIL
one thread executes
Python bytecode at a
time

The GIL constrains Python bytecode, not all native computation.

PARALLELISM | Numba ≈ OpenMP

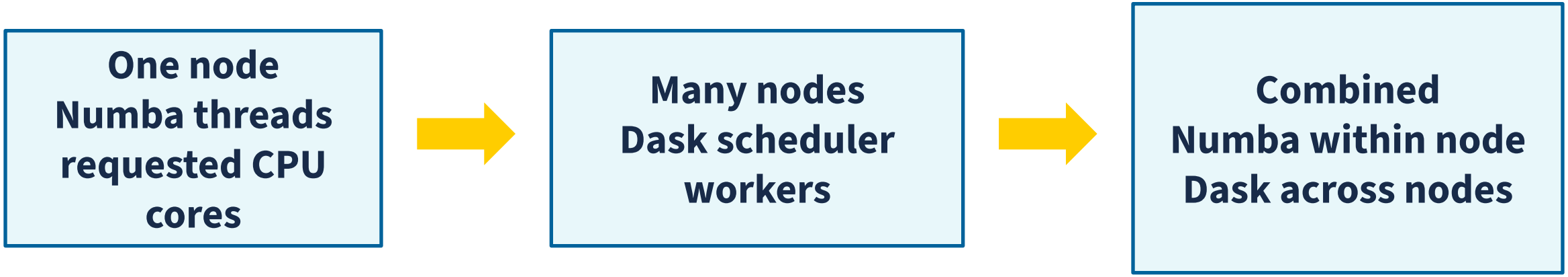
Python loop
serial
one core

Numba
`parallel=True`
`prange`

CPU cores
shared memory
all requested cores

Think of Numba as OpenMP-like: compile + `prange` **+ shared-memory threads on one node.**

PARALLELISM | One node vs many nodes



Workshop rule: Numba within a node; Dask across nodes.

SETUP | Launch Jupyter from the cache

- When staging finishes, run `bash launch_galyleo_compute.sh`.
- Galyleo stages `pythonhpc` from the cached archive.
- Open `README.md` and run the four-package import cell.
- Yellow = imports work. Blue = ask a helper.

2 of 7 | Numba serial

- Time check: 8:55 AM.
- Goal: speed up one serial, single-threaded numerical loop and time it fairly.
- Focus: establish the single-threaded baseline before adding parallelism.

NUMBA SERIAL | First call vs later calls

- Add `@jit` above a function to ask Numba to compile it.
- The first call compiles the function and returns an answer.
- Later calls reuse the compiled code.
- Verify the result, warm up once, then time later calls.

NUMBA SERIAL | Compile and benchmark

- Open folder `1_numba`.
- Open `0_basics.ipynb`.
- Run through "Compare with NumPy."
- Predict each timing before you run it.
- Stop at "Your turn."

NUMBA SERIAL | Check your mental model

- Did all three versions return the same value?
- Which timing includes compilation?
- Why do we call the function once before timing it?

NUMBA SERIAL | Hands-on

- Complete `sum_of_squares`.
- Check the result against NumPy.
- Warm up, then time both versions.
- 12 minutes. Blue means help. Yellow means ready.

NUMBA SERIAL | Debrief

- First call = compile + execute.
- Later calls = execute the compiled code.
- Warm up before benchmarking.
- Time check: 9:25 AM.

3 of 7 | Numba threads

- Time check: 9:25 AM.
- Goal: run one numerical loop across CPU cores with Numba.
- Focus: shared-memory parallelism across CPU cores.

NUMBA THREADS | Parallel loops across cores

- Open folder `1_numba`.
- Open `2_threads.ipynb`.
- Compare `range` with `prange`.
- Run serial and parallel versions, then inspect the speedup.
- Stop at "Your turn."

NUMBA THREADS | Check your mental model

- Why does the GIL limit CPU-bound Python threads?
- What changes when Numba compiles the loop and uses `prange`?
- How should thread count relate to CPUs requested from SLURM?

NUMBA THREADS | Hands-on

- Run the serial and `parallel=True` versions.
- Use `parallel_diagnostics()` to verify the loop parallelized.
- Relate Numba's thread count to the CPUs allocated by SLURM.
- 12 minutes. Blue means help. Yellow means ready.

NUMBA THREADS | Takeaway

- The GIL limits CPU-bound Python bytecode.
- Numba can run independent compiled iterations across CPU cores.
- Think Numba ≈ OpenMP: shared-memory parallelism within one node.
- Time check: 9:43 AM.

NUMBA | Real-world example: PySM

- PySM is an open-source CMB sky-simulation project I maintain.
- Its production model kernels rely on Numba for numerical performance.
- The dust-emission kernel uses `@njit(parallel=True)`.
- See `dust.py` lines 135–168 on GitHub.

4 of 7 | Break

- Time check: 9:45 AM.
- Break ends at 9:55 AM.
- Next: Dask task graphs and scaling.

5 of 7 | Dask

- Time check: 9:55 AM.
- Goal: see a task graph, then scale the same model across nodes.
- Focus: task graphs, lazy execution, and scaling.

DASK | A graph is a plan

- A task is one unit of work.
- A task graph records dependencies between tasks.
- The scheduler assigns ready tasks to workers.
- `compute()` triggers execution and returns the result.

DASK | Visualize a task graph

- Open folder `3_dask`.
- Open `0_dask_graphs.ipynb`.
- Build the expression, visualize its graph, then compute the result.
- Complete "Your turn" using the two provided variants.
- Predict how the task graph will change before each run.

DASK | Lazy execution with delayed

- Open folder `3_dask`.
- Open `1_delayed.ipynb`.
- Run through "Delay calls, then compute once."
- Inspect what exists before `compute()`.
- Stop at "Your turn."

DASK | Lazy execution: takeaway

- Put the final summary inside the graph.
- Call `compute()` once, on the final result.
- 5 minutes. Blue means help. Yellow means ready.
- Explain what existed before and after `compute()`.

DASK | Arrays become task graphs

- In folder `3_dask`, open `2_multicore_array.ipynb`.
- Run through "Choose chunks."
- Focus on how array operations become scheduled tasks.
- Change the partitioning once, then stop at "Your turn."

DASK | Debrief

- A graph is a plan; `compute()` triggers execution.
- Dask arrays turn array operations into scheduled tasks.
- Within one node we prefer Numba; next we use Dask across nodes.
- Time check: 10:30 AM.

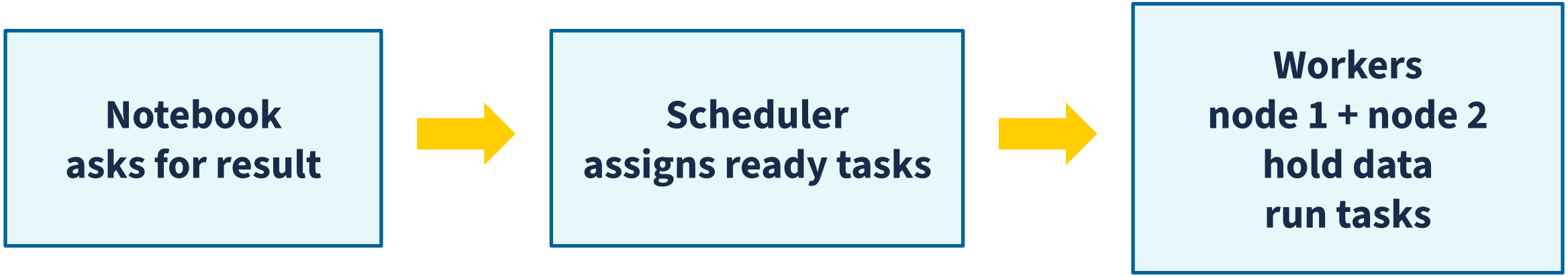
CLUSTER SETUP | Start scheduler and workers

- Time check: 10:30 AM.
- One JupyterLab terminal: `cd 6.1_python_for_HPC`
- `bash dask_slurm/launch_scheduler.sh &`
- `sbatch dask_slurm/dask_workers.slrm`
- `squeue --me` → continue when two workers connect.

6 of 7 | Multi-node Dask

- Time check: 10:40 AM.
- Goal: run one Dask expression on two worker nodes.
- Focus: distribute the same computation across multiple nodes.

MULTI-NODE DASK | Three roles



The same Dask expression runs against the distributed cluster.

MULTI-NODE DASK | Connect to the cluster

- Open folder `3_dask`.
- Open `4_multinode_distributed_array.ipynb`.
- Run "Start the cluster."
- Open the Dask dashboard and confirm two worker hosts.
- Stop before "Build an array across the workers."

MULTI-NODE DASK | Run across two nodes

- Build the array expression and compute it across both workers.
- Inspect worker hosts, threads, and the result.
- Confirm the expected value.
- 18 minutes. Blue means help. Yellow means ready.

MULTI-NODE DASK | Debrief and clean up

- What changed when we added nodes?
- What stayed the same in the Python expression?
- Cancel the worker job and stop the background scheduler.
- Time check: 11:05 AM.

7 of 7 | Recap

- Time check: 11:05 AM.
- Goal: choose the smallest parallel layer that solves the problem.
- Final questions end at 11:20 AM.

RECAP | Which layer should you use?

- CPU-bound Python loop: compile it with Numba.
- One node: Numba threads across CPU cores.
- Many nodes: Dask distributes tasks across workers.
- Combined: Dask across nodes + Numba within each node.

RECAP | What you take home

- Reusable notebooks and SI26 launch templates.
- A clear rule for one-node vs multi-node parallelism.
- A workflow you can adapt to your own scientific code.
- Time check: 11:20 AM. Questions?

OPTIONAL | AI coding assistants

AI AGENTS | Close the execution loop

- Give the agent a concrete goal and let it edit.
- Let it run the program or tests itself.
- It reads errors and output, fixes the code, and reruns.
- Once verified, inspect `git diff` before you push.

Autonomy needs a safety boundary

Grant broad permissions only inside a recoverable workspace that does not expose secrets.

OPENCODE | Bring your own model API

- OpenCode is the terminal agent/client — it does not include the model.
- What you need: LLM access via API — endpoint + credentials.
- Use your university's API; otherwise request NRP LLM access and use its OpenAI-compatible API for hosted open-weight models.
- NRP + OpenCode setup guide.

GITHUB COPILOT | Three interfaces

Inline completion while you type

VS Code agent edits files runs tools

Copilot CLI terminal agent runs commands

Use completion for small edits; use agents for multi-step work.

GITHUB COPILOT | Student access

- Verified GitHub Education students get Copilot Student at no cost.
- Code completions are unlimited on Copilot Student.
- Chat, agent mode, and Copilot CLI use the monthly AI Credits allowance.
- Check your usage dashboard before a long agent session.